PYTHON LIBRARIES

FOR MACHINE LEARNING ENGINEERS

Production-ready guide | AWS + GCP | Interview preparation

Real code, line-by-line explanations, scenario questions and senior-level practice tasks

12
TOP LIBRARIES
20 pages | ML role only
AWS + GCP

Deepika Chintala
LinkedIn: linkedin.com/in/deepikachintala

WHERE EACH LIBRARY LIVES IN A PRODUCTION ML PIPELINE

1 DATA	2 TRAIN	3 TRACK	4 ORCHESTRATE	5 SERVE + AUTOMATE
Get and clean data NumPy, Pandas, PySpark	**Teach the model** scikit-learn, XGBoost, PyTorch, TensorFlow	**Record the results** MLflow	**Run steps on a schedule** Airflow, Vertex AI + KFP	**Use it live, automate** FastAPI, boto3

1 NumPy | DATA | p.2-3
What Fast math on big lists of numbers.
Where Data step. It sits underneath Pandas, scikit-learn and PyTorch.
Example Scale 1 million payment amounts to the same range in one line, with no loop.

2 Pandas | DATA | p.3-4
What Excel for Python: tables you can clean and combine with code.
Where Data step. Clean the raw file and build the columns a model needs.
Example Total each customer's spending from a payments file and save the result.

3 scikit-learn | TRAIN | p.5-6
What Ready-made models for table data, like: will this customer leave?
Where Training step. Build your first simple model and a baseline score.
Example Predict customer churn from age, plan and monthly usage.

4 XGBoost | TRAIN | p.6-7
What A very accurate model made of many small decision trees.
Where Training step. The go-to model when your data is a table.
Example Decide in milliseconds whether a card payment is fraud or normal.

5 PyTorch | TRAIN | p.8-9
What A toolkit for neural networks (deep learning), loved by researchers.
Where Training step, for images, speech and text. Runs fast on GPUs (graphics chips).
Example Sort scanned cheques and documents into types for a bank.

6 TensorFlow | TRAIN + SERVE | p.9-10
What Google's deep learning toolkit, strong at getting models into real apps.
Where Train the model, then ship it to servers, browsers or phones.
Example Show product recommendations on a shopping app in a blink.

7 PySpark | DATA | p.11-12
What Python for Spark: many computers share one huge data job.
Where Data step, when the data is too big for one computer.
Example Turn billions of daily clicks into one summary row per user overnight.

8 Airflow | ORCHESTRATE | p.12-13
What A scheduler that runs your steps in order and retries when one fails.
Where The glue. It starts the data, training and deploy steps on time.
Example Every night at 2 AM: get data, train, check the score, deploy.

9 MLflow | TRACK | p.14-15
What A notebook that records every experiment: settings, scores and models.
Where After training. Compare runs and keep the approved model.
Example Find which of 30 trials scored best, or roll back to last month's model.

10 boto3 | AUTOMATE ON AWS | p.15-16
What A Python remote control for AWS: S3 files, SageMaker jobs and more.
Where Automation. Move files, start training jobs and clean up.
Example A nightly script that deletes unused test servers and cuts the bill.

11 Vertex AI SDK + KFP | ORCHESTRATE ON GCP | p.17-18
What Google Cloud tools to build ML steps and run them as one pipeline.
Where Runs data, train, check and deploy as one automatic flow on GCP.
Example A weekly retrain that deploys the new model only if its score improves.

12 FastAPI | SERVE | p.18-19
What Turns a trained model into a web address that apps can call.
Where The last step. Users and apps get live predictions here.
Example A payment app sends a transaction and gets a fraud score back in under 100 ms.

1 NumPy

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture A shop with 1 million price tags. You must add tax to every tag.

Trouble Walking to each tag with a pencil takes forever. That is a Python loop.

Helper NumPy is a stamping machine. It updates all the tags at the same moment.

In ML One line of math changes a whole array of numbers at once.

WHY THIS LIBRARY

- Every ML library (Pandas, scikit-learn, PyTorch) sits on top of NumPy arrays.
- Vectorized math runs in compiled C code, often 10-100x faster than loops.
- Fixed dtype + contiguous memory means less RAM and no type surprises.
- Broadcasting scales or normalizes data without writing a single loop.

WHEN TO USE / WHEN TO SKIP

- **Use:** scaling, distance math, windows, masks, simulations, batch scoring.
- **Use:** converting model inputs/outputs (image batches, embeddings, scores).
- **Skip:** mixed-type labeled tables (Pandas) or data bigger than RAM (Spark).

HOW TO USE IT

1) Build arrays with `np.array`, `np.load` or `np.frombuffer` on S3/GCS bytes.
2) Pick the smallest safe dtype: float32 instead of float64 halves memory.
3) Replace each for-loop with vector ops, masks and axis reductions.
4) Print **shape** and **dtype** at every pipeline boundary.

ADVANTAGES

- Speed and memory efficiency on large numeric batches.
- Common language: every ML framework converts to and from ndarray.
- Tiny, rock-solid dependency; runs the same on Lambda, Cloud Run, EMR.
- Deterministic with a seeded generator, so tests are easy.

REAL-WORLD EXAMPLES (illustrative)

- **Amazon-style:** normalizing millions of item-demand vectors before SageMaker training.
- **Google-style:** dot-product similarity for candidate retrieval prototypes.
- **Meta-style:** image batches as float32 arrays before they become PyTorch tensors.
- **Tesla-style:** sensor and telemetry windows resampled and stacked as arrays.
- **Bank:** risk features (utilization ratios, z-scores) on card transactions.

PRODUCTION TASKS | AWS + GCP

- Scaling and clipping code shared by training AND serving (one function).
- Batch scoring in Lambda or Cloud Run Jobs with preallocated arrays.
- Reading binary feature files from S3/GCS with `np.frombuffer`, no temp disk.
- Parity tests: `np.allclose(offline_features, online_features)` in CI.

COST + TIME SAVED

- Swapping loops for vector ops often turns an overnight job into a coffee-break job: smaller instance, shorter AWS/GCP bill.
- float32 instead of float64 halves memory, so one node fits what needed two.

EXPERT VIEW

- If you catch yourself writing a for-loop over rows, stop and ask: what is the vector version of this?

1 NumPy

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```
1  import numpy as np
2  def zscore_batch(x, mean, std, clip=5.0):
3      x = x.astype(np.float32, copy=False)
4      z = (x - mean) / np.maximum(std, 1e-6)
5      return np.clip(z, -clip, clip)
6  rng = np.random.default_rng(42)
7  amt = rng.lognormal(3, 1, size=(1000, 4))
8  mean, std = amt.mean(axis=0), amt.std(axis=0)
9  scaled = zscore_batch(amt, mean, std)
10 bad = (np.abs(scaled) > 3).any(axis=1)
11 print(scaled.shape, bad.sum())
```

INTERVIEW CODE QUESTION

- Replace the per-row loop with a vectorized cosine similarity. Why is the loop slow, and what changes in the fast version?

THE INTERVIEW CODE

```
def cosine_loop(M, q):
    out = []
    for row in M:
        out.append(row @ q / (np.linalg.norm(row)
                   * np.linalg.norm(q)))
    return np.array(out)

def cosine_fast(M, q):
    n = np.linalg.norm(M, axis=1)
    return (M @ q) / (n * np.linalg.norm(q) + 1e-12)
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

- **L1** Standard alias **np**: every team uses it, so reviews stay readable.
- **L2** One scaling function imported by training AND serving: no train/serve skew.
- **L3** float32 halves memory; **copy=False** skips a copy if it already is float32.
- **L4** Broadcasting: per-column mean/std applied to every row. **np.maximum** prevents divide-by-zero.
- **L5** Clip tames wild values so one bad transaction cannot wreck the model input.
- **L7** **default_rng(42)**: reproducible random numbers (modern Generator API).
- **L8** Fake lognormal money amounts: 1000 rows x 4 features.
- **L9** **axis=0** runs down the rows, giving one mean and std per column.
- **L10** Same function we would call in production.
- **L11** Mask: any value beyond 3 sigma in the row. **axis=1** collapses across columns.
- **L12** Prints (1000, 4) and the flagged-row count. Always eyeball the shape.

INTERVIEW CODE | STEP BY STEP

1) The loop calls Python and recomputes **norm(q)** once per row, millions of times.
2) **M @ q** is one matrix-vector product: every dot product in a single C/BLAS call.
3) **norm(M, axis=1)** gives all row norms at once.
4) Element-wise divide of two arrays; the epsilon avoids 0/0 for all-zero rows.
5) Same O(n*d) math, but running in optimized C: often 50-100x faster.
6) Follow-up they ask: data larger than RAM? Process chunks or use **np.memmap**.

1 NumPy

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: Nightly batch scoring takes 3 hours and keeps hitting out-of-memory. Profiling shows float64 arrays and Python loops.

1) Measure first: line_profiler and memory_profiler. Never guess.
2) Downcast to float32/int32 where safe; verify with `np.allclose` on a sample.
3) Vectorize loops; replace `np.append` in loops (O(n squared)) with preallocation.
4) Chunk the work (`np.array_split`) or use `np.memmap` if data exceeds RAM.
5) Result: same logic on a smaller instance, lower bill. Document the dtype contract.

SCENARIO Q2 | PRODUCTION

Situation: A bank's fraud model scores differ slightly between offline training and the online API. Where do you look?

1) Classic train/serve skew. First suspect: scaling code copy-pasted in two places.
2) Move preprocessing into one shared package imported by both paths.
3) Check dtype differences (float64 offline, float32 online) and NaN handling.
4) Parity test: same 1,000 rows through both paths, assert `np.allclose(rtol=1e-5)`.
5) Run it in CI before every deploy so skew never reaches customers.

SCENARIO Q3 | PRODUCTION

Situation: Someone computes rolling averages over 10 years of per-second sensor data with a Python loop. It runs for hours.

1) Vectorize with a cumulative-sum trick or sliding windows instead of a loop.
2) Process in chunks with (window - 1) rows of overlap so boundaries stay correct.
3) Check against the loop on a small sample with `np.allclose` before trusting it.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Build a vectorized rolling-window feature generator (mean, std, z-score); benchmark vs a loop.
2) Write a memmap-based chunked scorer for a 50 GB feature file on a 16 GB instance.
3) Create a parity-test harness (offline vs online features) with tolerances, run in CI.
4) Implement top-k cosine retrieval with `argpartition` instead of a full sort; compare latency.
5) Package a dtype-contract validator (shape, dtype, NaN, range) used by Lambda and Cloud Run services.

KEY POINTS TO REMEMBER IN INTERVIEWS

- Vectorize first, loop last. Broadcasting aligns shapes from the right.
- Slicing returns a **view**, fancy indexing returns a **copy**. Editing a view changes the original!
- **axis=0** goes down rows, **axis=1** goes across columns.
- Never compare floats with ==; use `np.isclose` / `np.allclose`.
- Seed with `default_rng`; log the seed with the experiment.
- NaN poisons sum/mean: use `nanmean` / `nansum` or clean upstream.

COMMON MISTAKES THAT COST OFFERS

- Python lists of lists for numeric work, then wondering why it is slow.
- Editing a view and silently changing the source array.
- Comparing floats with ==.
- Leaving everything float64 on a memory-starved instance.

2 Pandas

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture A huge notebook of customer rows: name, city, amount paid. Some cells are blank, some have typos.

Trouble Fixing it by hand takes days, and nobody can repeat it exactly.

Helper Pandas sorts, filters, fills blanks and totals each customer in seconds. Every step is code, so you can replay it.

In ML Pandas turns messy raw data into a clean table a model can learn from.

WHY THIS LIBRARY

- The standard tool for tabular cleaning and feature engineering.
- Reads CSV, Parquet, JSON and SQL straight from S3, GCS, BigQuery, Redshift.
- A readable chain of transformations that teammates can review.
- Plugs directly into scikit-learn, XGBoost and PyTorch dataloaders.

WHEN TO USE / WHEN TO SKIP

- **Use:** data that fits comfortably in RAM; EDA, feature tables, validation checks.
- **Use:** small batch jobs in Lambda or Cloud Run Jobs.
- **Skip:** tens of GB or more (PySpark, BigQuery) and row-by-row serving paths.

HOW TO USE IT

1) Read with explicit dtypes and **usecols**; prefer Parquet over CSV.
2) Clean: types, nulls, duplicates, outliers, as a chain of small pure functions.
3) Engineer: groupby/agg, merge, rolling windows, categories.
4) Validate schema and null rates, then write Parquet back to S3/GCS.

EXPERT VIEW

- Pandas is for data that fits in memory comfortably. The day it does not, move the logic to Spark or SQL, not just to a bigger machine.

ADVANTAGES

- Huge ecosystem: every teammate already knows it.
- Fast enough for most tabular work because NumPy does the heavy lifting.
- Rich time-series tools: resample, rolling, shift, timezones.
- Parquet + categorical dtypes cut memory dramatically.

REAL-WORLD EXAMPLES (illustrative)

- **Amazon-style:** seller and order feature tables built before SageMaker Pipelines training.
- **Google-style:** BigQuery results pulled with `to_dataframe()` for model prototyping.
- **Meta-style:** offline analysis of A/B experiment results for ranking models.
- **Tesla-style:** telemetry and log resampling and cleaning before model training.
- **Bank:** customer-360 features: transaction counts, average ticket, days since login.

PRODUCTION TASKS | AWS + GCP

- Feature engineering steps in SageMaker Processing and Vertex AI Pipelines components.
- Data validation gates: null rates, schema drift, duplicates before training.
- Scheduled data-quality scripts replacing manual Excel checks.
- Offline evaluation tables: predictions joined with ground truth for metrics.

COST + TIME SAVED

- CSV to typed Parquet often shrinks files several times over and speeds reads, cutting S3/GCS storage and scan costs.
- Automated data-quality checks replace a daily manual analyst hour and catch bad data before training burns GPU money.

2 Pandas

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```python
import pandas as pd
df = pd.read_parquet("s3://ml-bank/txns/2026-10-08/",
                     columns=["cust_id", "ts", "amount"])
df["ts"] = pd.to_datetime(df["ts"], utc=True)
df = (df.drop_duplicates(["cust_id", "ts"])
        .query("amount > 0"))
feat = (df.groupby("cust_id")
          .agg(n_txn=("amount", "size"),
               avg_amt=("amount", "mean"),
               last_seen=("ts", "max")))
feat.to_parquet("s3://ml-bank/features/cust.parquet")
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

L1 Standard alias **pd**.

L2-3 Read straight from S3 (needs **s3fs**). **columns=** loads 3 columns only: less RAM, less I/O.

L4 Parse text into real UTC datetimes; strings sort wrongly and break time math.

L5-6 Drop duplicate events (retries happen!) and invalid amounts. The chain reads top to bottom.

L7-10 Named aggregation: one row per customer with count, mean and latest time. That is a feature table.

L11 Write Parquet back: typed, compressed and ready for training.

INTERVIEW CODE QUESTION

- For each customer, find the top 3 merchants by total spend. Write it and explain every step.

INTERVIEW CODE | STEP BY STEP

1) Group by customer + merchant and sum amount: spend per pair.

2) **as_index=False** keeps flat columns, no MultiIndex pain later.

3) Sort by customer, then spend descending, so best merchants come first.

4) **groupby("cust_id").head(3)** keeps the first 3 rows per customer: the top 3.

5) Alternatives: **nlargest** per group, or **rank(method="first")** and filter.

6) Follow-up: 500M rows? Say PySpark: **row_number()** over a window partitioned by customer.

THE INTERVIEW CODE

```python
top3 = (df.groupby(["cust_id", "merchant"],
                   as_index=False)["amount"].sum()
          .sort_values(["cust_id", "amount"],
                       ascending=[True, False])
          .groupby("cust_id").head(3))
```

2 Pandas

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: A preprocessing step crashes with MemoryError on a 20 GB file inside a SageMaker Processing job.

1) Check dtypes first: object to category, float64 to float32, int64 to int32.

2) Load only needed columns and push filters to the source (Parquet filters, Athena, BigQuery).

3) Process in chunks or per-day partitions (`chunksize=` for CSV, one Parquet file at a time).

4) If data keeps growing, do not fight it: move that step to PySpark or BigQuery.

5) Say the trade-off aloud: a bigger instance is a quick fix, not a design.

SCENARIO Q2 | PRODUCTION

Situation: After a merge, the customer table has twice as many rows and the features look wrong.

1) Duplicate keys on the right table: a many-to-many join multiplies rows.

2) Use `merge(..., validate="many_to_one")` so it fails loudly.

3) Assert row counts before and after in the pipeline's data-quality gate.

4) Check key dtypes (str vs int) and whitespace; they cause silent misses too.

5) Fix dedup upstream and add a unit test containing a duplicated key.

SCENARIO Q3 | PRODUCTION

Situation: A teammate's notebook uses `iterrows()` over 10M rows and takes hours.

1) Replace it with vectorized column math or `np.where`; iterrows is a last resort.

2) Use `groupby(...).transform(...)` for per-group math instead of loops.

3) If the logic is truly custom, move to Spark or Numba, and report the speed-up in the PR.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Write a reusable data-quality module (null %, duplicates, schema, ranges) that fails the pipeline with a readable report.

2) Convert a 5 GB CSV flow to chunked Parquet; measure memory and runtime before and after.

3) Build point-in-time-correct features with `merge_asof` so no future data leaks into training.

4) Automate a weekly drift report (PSI per feature) and post it to Slack from Lambda or Cloud Run.

5) Refactor a 400-line notebook into tested pure functions chained with `DataFrame.pipe`.

KEY POINTS TO REMEMBER IN INTERVIEWS

- SettingWithCopyWarning: use `.loc[mask, col] = value` or work on `.copy()`.
- `apply(axis=1)` is slow: prefer vectorized ops, `np.where`, `map`.
- Merge: always think about key uniqueness; use `validate=`.
- Use **category** dtype for low-cardinality strings.
- Time series: sort first; avoid look-ahead leakage in rolling and shift.
- `inplace=True` does not save memory; prefer reassignment and chaining.

COMMON MISTAKES THAT COST OFFERS

- Chained assignment like `df[a][b] = x`.
- Reading a whole CSV just to use 3 columns.
- Leaving low-cardinality text as object dtype.
- Joining without checking duplicate keys.

3 scikit-learn

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture A cooking class where every recipe card has the same 3 steps: choose, practice, cook for a guest.

Trouble Learning a new set of steps for every dish is slow and confusing.

Helper scikit-learn gives every model the same steps: create it, fit it (practice on examples), predict (cook for a new guest). A Pipeline is the full recipe card.

In ML The same simple commands work for many models, and cleaning steps travel with the model.

WHY THIS LIBRARY

- One consistent API (fit / predict / transform) for 100+ classic algorithms.
- **Pipeline** + **ColumnTransformer** keep preprocessing and model in a single object.
- Cross-validation, grid search and metrics are built in.
- Baselines in minutes: logistic regression, random forest, gradient boosting.

WHEN TO USE / WHEN TO SKIP

- **Use:** tabular classification/regression, clustering, preprocessing, baselines.
- **Use:** when interpretability and fast retraining matter (banking, credit).
- **Skip:** deep nets on images/text (PyTorch, TF) or distributed training (Spark).

HOW TO USE IT

1) Split first (time-based for time series).
2) Build a **ColumnTransformer**: impute, scale, one-hot.
3) Wrap with the model in a **Pipeline**; cross-validate.
4) Save with **joblib**; load inside the serving container.

ADVANTAGES

- Simple, stable and well documented: fewer production surprises.
- Pipeline prevents leakage: transformers fit only on training folds.
- Runs on SageMaker (script mode) and Vertex AI custom containers.
- Easy to explain to risk, audit and compliance teams.

REAL-WORLD EXAMPLES (illustrative)

- **Amazon-style:** quick churn and delivery-delay baselines before heavier models.
- **Google-style:** tabular baselines on BigQuery data before scaling out.
- **Meta-style:** logistic regression as a sanity-check baseline against deep models.
- **Tesla-style:** anomaly detection on component test data with IsolationForest.
- **Bank:** credit-risk scorecards: logistic regression plus calibration for regulators.

PRODUCTION TASKS | AWS + GCP

- Training jobs on SageMaker / Vertex AI with joblib artifacts saved to S3 / GCS.
- Scheduled retraining with metric gates: deploy only if AUC beats the champion.
- Batch scoring jobs; real-time endpoints behind FastAPI.
- RandomizedSearchCV with fixed seeds, every run logged to MLflow.

COST + TIME SAVED

- Classic models train in minutes on CPU: no GPU bill, and on many tabular problems they match deep models.
- One Pipeline artifact replaces separate scaling scripts: fewer bugs and fewer 2 a.m. pages.

EXPERT VIEW

- The model is 20% of the work. The split strategy, the Pipeline and the metric are the other 80%.

3 scikit-learn

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```python
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
import joblib

num = Pipeline([("imp", SimpleImputer(strategy="median")),
                ("sc", StandardScaler())])
pre = ColumnTransformer([("num", num, ["income", "age"]),
  ("cat", OneHotEncoder(handle_unknown="ignore"), ["channel"])])
model = Pipeline([("pre", pre), ("clf",
    RandomForestClassifier(200, random_state=7, n_jobs=-1))])
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
joblib.dump(model, "credit_model.joblib")
```

INTERVIEW CODE QUESTION

- This code reports a great score but fails in production. Find the leak and fix it.

THE INTERVIEW CODE

```python
# BAD: the scaler saw the test rows
X_s = StandardScaler().fit_transform(X)
X_tr, X_te, y_tr, y_te = train_test_split(X_s, y)
# GOOD: split first, fit inside a Pipeline
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, stratify=y, random_state=7)
pipe = Pipeline([("sc", StandardScaler()),
                 ("clf", clf)])
pipe.fit(X_tr, y_tr)
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

L1-6 Imports: pipeline, column routing, imputer, scalers, model and joblib to save it.

L7-8 Numeric recipe: fill missing with the median (robust to outliers), then scale.

L9-10 Route columns: numbers get the recipe, categories get one-hot. **handle_unknown="ignore"** stops crashes on new categories in production.

L11-12 Glue preprocessing + model into ONE object. **random_state** makes runs repeatable; **n_jobs=-1** uses all cores.

L13 fit learns the medians, scaling AND the forest from training data only: no leakage.

L14 Score on held-out data (accuracy here; use AUC or PR-AUC for imbalance).

L15 One artifact holds everything; serving only calls **.predict_proba()**.

INTERVIEW CODE | STEP BY STEP

1) The bad version fits the scaler on ALL rows, so test-set mean and std leak into training.
2) Offline metrics look better than reality; production AUC then drops.
3) Fix: split first, fit transformers only on the training part.
4) **Pipeline** guarantees this, even inside **cross_val_score** (refits per fold).
5) **stratify=y** keeps the class ratio, vital for imbalanced fraud data.
6) Follow-up: time data? Use **TimeSeriesSplit**; a random split leaks the future.

3 scikit-learn

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: Fraud data has 0.2% positives. Accuracy is 99.8% yet the model catches nothing.

1) Accuracy is meaningless here: always answering 'not fraud' scores 99.8%.
2) Switch metrics: PR-AUC, recall at fixed precision, or cost-based.
3) Use `class_weight="balanced"` or resample inside CV folds only.
4) Tune the decision threshold to business cost: missed fraud vs blocked customer.
5) Calibrate probabilities (`CalibratedClassifierCV`) so risk teams can trust the scores.

SCENARIO Q2 | PRODUCTION

Situation: Offline AUC was 0.91; two months after deployment it is 0.78. How do you debug?

1) Rule out leakage first: re-check feature timestamps and split strategy.
2) Then drift: compare live vs training feature distributions (PSI, KS test).
3) Check upstream schema or unit changes, new categories, missing rates.
4) Retrain on a recent window behind a champion/challenger gate.
5) Add monitoring: weekly AUC on a labeled sample plus drift alerts.

SCENARIO Q3 | PRODUCTION

Situation: A teammate tunes hyperparameters on the test set and reports 0.95 AUC.

1) That is leakage by selection: the test set is touched once, at the very end.
2) Tune with nested CV or a separate validation set.
3) Log every tuning run in MLflow so the process is auditable.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Build a reusable training pipeline with ColumnTransformer and a custom transformer, tested with pytest.
2) Implement time-based cross-validation; compare with random CV to quantify leakage.
3) Write a champion/challenger script: retrain, compare PR-AUC, promote only if better.
4) Add probability calibration and cost-matrix-driven threshold selection.
5) Containerize for SageMaker (`model_fn`, `predict_fn`) and deploy to an endpoint.

KEY POINTS TO REMEMBER IN INTERVIEWS

- Fit on train only; **Pipeline** makes leaks impossible.
- Know **fit** vs **transform** vs **fit_transform**.
- Imbalanced data: PR-AUC, class_weight, threshold tuning.
- joblib pickles are version-sensitive: pin sklearn in training AND serving images.
- Set `random_state` everywhere and log hyperparameters.
- `cross_val_score` returns per-fold scores: report mean and std.

COMMON MISTAKES THAT COST OFFERS

- Scaling before the split.
- Accuracy as the metric for rare events.
- Not pinning library versions for joblib files.
- Random split on time-dependent data.

4 XGBoost

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture 500 students take the same exam, one after another.

Trouble One student alone gets many answers wrong.

Helper Student 2 studies only what student 1 missed. Student 3 studies what is still wrong, and so on. Each student is one small decision tree.

In ML Boosting: many small trees, each one fixing the mistakes of the trees before it.

WHY THIS LIBRARY

- Often best-in-class on tabular data; a staple in banking and risk.
- Handles missing values natively: no heavy imputation.
- Built-in regularization (lambda, alpha, gamma) fights overfitting.
- Fast: parallel tree building and optional GPU (`device="cuda"`).

WHEN TO USE / WHEN TO SKIP

- **Use:** fraud, credit risk, churn, ranking, demand forecasting on tables.
- **Use:** strong accuracy with modest tuning effort.
- **Skip:** images, audio, text (deep nets) or when strict linear interpretability is mandated.

HOW TO USE IT

1) Prepare features (numeric or categorical dtype).
2) Train with **early stopping** on a validation set.
3) Evaluate with PR-AUC / AUC and explain with SHAP.
4) Save with **save_model** and serve from a container.

EXPERT VIEW

- Fix the validation strategy before you tune hyperparameters. A leaky split beats any tuning.

ADVANTAGES

- Excellent accuracy-to-effort ratio on structured data.
- Early stopping finds the sweet spot and saves training time.
- Feature importance and SHAP support for explainability.
- Built-in algorithm on SageMaker; prebuilt containers on Vertex AI.

REAL-WORLD EXAMPLES (illustrative)

- **Amazon:** SageMaker built-in XGBoost is a common route for tabular models.
- **Google:** Vertex AI serving containers for XGBoost tabular jobs.
- **Meta-style:** gradient-boosted trees as strong feature-table baselines.
- **Tesla-style:** failure and maintenance prediction from tabular test and service data.
- **Bank:** fraud detection and credit-default scoring, often the champion model.

PRODUCTION TASKS | AWS + GCP

- Hyperparameter tuning jobs (SageMaker Automatic Model Tuning, Vertex AI Vizier).
- Nightly retrain with early stopping and automatic champion comparison.
- Batch inference on millions of rows; low-latency endpoints for fraud checks.
- SHAP reason codes stored with each decision for audits.

COST + TIME SAVED

- Early stopping stops paying for rounds that no longer help, often a large share of the budget.
- CPU-friendly inference: millisecond scoring on small instances keeps endpoint bills low.

4 XGBoost

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```python
import xgboost as xgb
from sklearn.metrics import average_precision_score

dtrain = xgb.DMatrix(X_tr, label=y_tr)
dval = xgb.DMatrix(X_val, label=y_val)
params = {"objective": "binary:logistic",
          "eval_metric": "aucpr", "eta": 0.05,
          "max_depth": 6, "subsample": 0.8,
          "colsample_bytree": 0.8,
          "scale_pos_weight": 99}
bst = xgb.train(params, dtrain, num_boost_round=2000,
                evals=[(dval, "val")],
                early_stopping_rounds=50)
best = (0, bst.best_iteration + 1)
p = bst.predict(dval, iteration_range=best)
print(average_precision_score(y_val, p))
bst.save_model("fraud_xgb.ubj")
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

- **L1-2** XGBoost core API plus PR-AUC, the right metric for rare fraud.
- **L3-4** **DMatrix** is XGBoost's optimized container. Keep a separate validation set.
- **L5-6** Binary fraud problem judged by aucpr. **eta=0.05** is a small learning rate: slower but safer.
- **L7-8** Subsample rows and columns per tree: randomness reduces overfitting.
- **L9** **scale_pos_weight** is roughly negatives/positives (99 means 1% fraud).
- **L10-12** Up to 2000 rounds, but stop when val aucpr has not improved for 50 rounds.
- **L13-14** Predict with the best iteration only, not the overfit trailing rounds.
- **L15** Report average precision on the validation set.
- **L16** Save in portable UBJ/JSON format: safer across versions than pickle.

INTERVIEW CODE QUESTION

- Training AUC is 0.99 but validation AUC is 0.80. What do you change in the code?

THE INTERVIEW CODE

```python
params.update({
    "max_depth": 4,
    "min_child_weight": 10,
    "gamma": 1.0,
    "reg_lambda": 5,
    "eta": 0.03})
# keep early_stopping_rounds on
```

INTERVIEW CODE | STEP BY STEP

1) A gap like this is overfitting: the model memorized training rows.
2) Shallower trees (**max_depth 4**) reduce complexity.
3) **min_child_weight** and **gamma** block splits on tiny, noisy groups.
4) **reg_lambda** / **reg_alpha** penalize large leaf weights (L2 / L1).
5) Lower eta + more rounds + early stopping = smoother learning.
6) Also check leakage (a feature secretly containing the label) before tuning anything.

4 XGBoost

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: A bank needs reason codes for every declined loan from an XGBoost model.

1) Use SHAP **TreeExplainer**: per-prediction feature contributions.
2) Map the top contributors to plain-language reason codes agreed with compliance.
3) Store SHAP values with the decision ID in S3 / BigQuery for audits.
4) Global importance alone does not explain one customer's decision.
5) Confirm sensitive or proxy features are excluded (fair lending).

SCENARIO Q2 | PRODUCTION

Situation: The real-time fraud endpoint must answer in under 50 ms, but scoring is slow.

1) Profile first: is the time in feature fetch, network or `predict()`?
2) Reduce trees and depth through tuning; drop low-gain features.
3) Load the model once at container start, never per request.
4) Use `inplace_predict` on numpy arrays; skip per-call pandas overhead.
5) Autoscale the endpoint (SageMaker / Cloud Run) and batch micro-requests.

SCENARIO Q3 | PRODUCTION

Situation: Adding `days_since_chargeback` boosts AUC, but that value is only known after fraud is confirmed.

1) That is target leakage: the feature is created after the label.
2) Compare each feature's availability time against prediction time.
3) Remove it, retrain, and expect the honest (lower) metric. Add this review to the model checklist.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Tune with Optuna or SageMaker AMT on PR-AUC with early stopping; track runs in MLflow.
2) Write a SHAP reason-code service that returns the top 3 reasons per prediction.
3) Compare XGBoost vs LightGBM vs logistic regression with time-split CV and a cost-of-error report.
4) Build a champion/challenger retraining DAG that gates promotion on PR-AUC and calibration.
5) Benchmark CPU vs GPU training on 50M rows and report dollars per run.

KEY POINTS TO REMEMBER IN INTERVIEWS

- Low **eta** + early stopping is the standard recipe.
- **scale_pos_weight** ~ negatives / positives for imbalance.
- Missing values are handled natively: do not impute blindly.
- Pickle is risky: use `save_model` and pin the xgboost version.
- Big train/val gap = overfit: reduce depth, add regularization.
- 'gain' vs 'weight' importance differ; prefer SHAP for explanations.

COMMON MISTAKES THAT COST OFFERS

- Tuning hyperparameters before fixing the validation split.
- Training a fixed 1000 rounds with no early stopping.
- Judging rare-fraud models by accuracy.
- Saving models with pickle across library versions.

5 PyTorch

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture Learning to ride a bike. You wobble, feel which way you lean, correct a little, try again.

Trouble A model cannot see its own mistakes unless something measures them.

Helper PyTorch guesses (wobble), measures the error (loss), finds which way to fix it (gradient) and nudges its numbers. You can pause and look inside with a normal print().

In ML Guess, measure the mistake, adjust, repeat many times.

WHY THIS LIBRARY

- Define-by-run graphs: debugging uses normal Python tools.
- Strong in research and increasingly in production (TorchScript, TorchServe, `torch.compile`).
- Huge ecosystem: torchvision, torchaudio, Lightning.
- First-class GPU and distributed training (DDP).

WHEN TO USE / WHEN TO SKIP

- **Use:** deep learning for vision, speech, recommendation, time series.
- **Use:** custom losses and architectures, fast experimentation.
- **Skip:** small tabular problems (XGBoost usually wins) or when a simple baseline is enough.

HOW TO USE IT

1) Dataset + **DataLoader** (batching, workers).
2) **nn.Module** model, loss function, optimizer.
3) Training loop: zero_grad, forward, loss, backward, step.
4) Evaluate under **no_grad()**; save state_dict; export TorchScript / ONNX.

EXPERT VIEW

- Make it work on one batch, then one GPU, then many GPUs. Skipping steps is how teams lose weeks.

ADVANTAGES

- Pythonic and easy to debug.
- Mixed precision (`autocast`) squeezes more out of each GPU.
- Scales out: DDP on multi-GPU and multi-node (SageMaker, Vertex AI).
- Large community: most new research ships PyTorch code.

REAL-WORLD EXAMPLES (illustrative)

- **Meta:** created PyTorch; used across vision and recommendation models.
- **Tesla:** has talked publicly about training its vision networks in PyTorch.
- **Amazon:** SageMaker PyTorch containers and TorchServe (built with AWS).
- **Google:** Vertex AI custom training accepts PyTorch containers.
- **Bank:** cheque and document image classification, sequence anomaly detection.

PRODUCTION TASKS | AWS + GCP

- Distributed training jobs (SageMaker, Vertex AI) with checkpoints saved to S3 / GCS.
- Mixed precision + gradient accumulation to fit big batches on fewer GPUs.
- Export to TorchScript / ONNX; serve via TorchServe, Triton or a managed endpoint.
- Automated retraining, evaluation and canary rollout.

COST + TIME SAVED

- Mixed precision commonly gives a 1.5-3x speedup on modern GPUs: fewer GPU-hours per run.
- Spot / preemptible GPUs with checkpointing often cut training cost by more than half.

5 PyTorch

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```python
import torch, torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
dl = DataLoader(TensorDataset(X, y), batch_size=256,
                shuffle=True, num_workers=2)
model = nn.Sequential(nn.Linear(20, 64), nn.ReLU(),
                      nn.Linear(64, 1))
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
loss_fn = nn.BCEWithLogitsLoss()
for epoch in range(5):
    model.train()
    for xb, yb in dl:
        opt.zero_grad()
        loss = loss_fn(model(xb).squeeze(1), yb)
        loss.backward()
        opt.step()
torch.save(model.state_dict(), "model.pt")
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

L1-2 Core imports. **DataLoader** feeds mini-batches to the model.

L3-4 Batch of 256, reshuffled every epoch; 2 worker processes load data in parallel.

L5-6 Tiny network: 20 features, 64 hidden units, 1 logit. No sigmoid needed here.

L7-8 AdamW optimizer. **BCEWithLogitsLoss** applies the sigmoid inside (numerically stable).

L9-10 Epoch loop. **model.train()** switches on dropout / batch-norm training behaviour.

L11-13 Per batch: clear old gradients, forward pass, compute loss.

L14-15 **backward()** computes gradients; **step()** updates the weights.

L16 Save weights only (state_dict), not a pickled model: portable and safer. Labels must be float for this loss.

INTERVIEW CODE QUESTION

- This training loop runs but the loss does not fall and GPU memory keeps growing. Spot the bugs.

THE INTERVIEW CODE

```python
# BUG 1: gradients pile up (no zero_grad)
for xb, yb in dl:
    loss = loss_fn(model(xb), yb)
    loss.backward(); opt.step()
# BUG 2: keeps the whole graph alive
total += loss          # wrong
total += loss.item()   # right
# Evaluation
model.eval()
with torch.no_grad():
    preds = model(X_val)
```

INTERVIEW CODE | STEP BY STEP

1) Gradients accumulate by default: call **zero_grad()** every step (or accumulate on purpose).
2) **total += loss** holds the autograd graph, so memory grows each step. Use **.item()**.
3) **model.eval()** turns off dropout and uses running batch-norm stats.
4) **no_grad()** saves memory and time during inference.
5) Shape check: logits (N,1) vs labels (N,). Squeeze, and labels must be float.
6) Follow-up: resume training? Save optimizer + epoch + state_dict in a checkpoint.

5 PyTorch

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: Training on 4 GPUs in SageMaker is only 1.3x faster than 1 GPU.

1) Likely a data-loading bottleneck: raise `num_workers`, use `pin_memory`, prefetch.
2) Use DistributedDataParallel (DDP), not DataParallel: one process per GPU.
3) Check batch size per GPU: too small and communication dominates.
4) Add mixed precision (`autocast` + `GradScaler`) to speed math and shrink memory.
5) Profile with `torch.profiler` and `nvidia-smi` before changing anything.

SCENARIO Q2 | PRODUCTION

Situation: A spot GPU instance was reclaimed after 14 hours of training. How do you stop losing the run?

1) Checkpoint regularly: model, optimizer, scheduler, epoch and RNG state.
2) Write checkpoints to S3 / GCS, not local disk (SageMaker managed spot uses `checkpoint_s3_uri`).
3) On restart, detect the latest checkpoint and resume the loop automatically.
4) Make the data sampler resumable so no sample is silently skipped.
5) Result: cheaper spot capacity with identical results. Cost drops, quality stays.

SCENARIO Q3 | PRODUCTION

Situation: Validation loss turns into NaN after a few hundred steps.

1) Check inputs for NaN / inf and extreme values first.
2) Lower the learning rate and add gradient clipping (`clip_grad_norm_`).
3) With mixed precision, confirm `GradScaler` is used; briefly try `set_detect_anomaly(True)`.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Write a custom Dataset that streams Parquet shards from S3 / GCS with a worker-safe reader.
2) Convert a single-GPU loop to DDP; log scaling efficiency for 1, 2 and 4 GPUs.
3) Add mixed precision, gradient accumulation and checkpoint-resume; prove metrics match after a resume.
4) Export to TorchScript / ONNX and benchmark latency on a SageMaker vs Vertex AI endpoint.
5) Build an evaluation harness: per-slice metrics, calibration and a regression gate in CI.

KEY POINTS TO REMEMBER IN INTERVIEWS

- Order: zero_grad, forward, loss, backward, step.
- **train()** vs **eval()**; **no_grad()** for inference.
- Use `.item()` or `.detach()` when logging to avoid memory leaks.
- Save **state_dict**; load with `map_location` when moving CPU/GPU.
- Shapes and dtypes are the number-one bug source.
- Set seeds and deterministic flags when you need reproducibility.

COMMON MISTAKES THAT COST OFFERS

- Forgetting `model.eval()` before validation.
- Not zeroing gradients.
- Copying data to the GPU one sample at a time inside the loop.
- Saving whole model objects instead of the state_dict.

6 TensorFlow

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture A factory. Raw material goes in, a conveyor belt feeds the machines, a sealed box comes out ready to ship.

Trouble A model that works only in your notebook helps nobody.

Helper Raw material is your data. The belt is `tf.data`. The machines are the layers (Keras). The sealed box is the SavedModel.

In ML TensorFlow trains a model, packs it into one folder, and ships it to servers, browsers and phones.

WHY THIS LIBRARY

- Production-first: SavedModel, TF Serving, TFLite, TF.js.
- **tf.data** builds fast, scalable input pipelines.
- Strong fit with GCP: Vertex AI, TPUs, TFX.
- Keras makes common models short and readable.

WHEN TO USE / WHEN TO SKIP

- **Use:** production deep learning with strict serving needs, TPU work, edge via TFLite.
- **Use:** recommendation, vision, forecasting with TFX pipelines.
- **Skip:** quick research prototypes when your team already lives in PyTorch.

HOW TO USE IT

1) **tf.data** pipeline: read, map, cache, shuffle, batch, prefetch.
2) Keras model: Sequential or Functional API.
3) **compile** and **fit** with callbacks (EarlyStopping, ModelCheckpoint).
4) Export SavedModel, then serve on TF Serving or a Vertex AI endpoint.

EXPERT VIEW

- Ship preprocessing inside the model. Training/serving skew hides in the gaps between two code paths.

ADVANTAGES

- Mature deployment story: TF Serving, TFLite, Vertex AI prediction.
- **tf.distribute** strategies: multi-GPU, multi-worker, TPU.
- TensorBoard for experiment visualization.
- TFX, TFDV and TFT keep validation and transforms consistent.

REAL-WORLD EXAMPLES (illustrative)

- **Google:** created TensorFlow; it runs across products and Vertex AI.
- **Amazon:** SageMaker TensorFlow containers and TensorFlow Serving endpoints.
- **Meta:** mainly a PyTorch shop, so 'TF vs PyTorch?' is a favorite question. Answer with balance.
- **Tesla-style edge:** small quantized models on constrained hardware, the TFLite way of thinking.
- **Bank:** TF Serving for real-time fraud and recommendations with strict latency SLAs.

PRODUCTION TASKS | AWS + GCP

- Vertex AI Pipelines / TFX: ingest, validate, transform, train, evaluate, push.
- TensorFlow Data Validation catches schema drift and skew before training.
- SavedModel to a Vertex AI endpoint (autoscaling) or a SageMaker endpoint.
- Quantization and TFLite conversion for edge and mobile.

COST + TIME SAVED

- TPUs/GPUs with `tf.distribute` shorten big training runs.
- `tf.data` prefetch keeps accelerators busy: you pay for compute, not for waiting.
- Quantization can shrink models around 4x with small accuracy loss: cheaper serving.

6 TensorFlow

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```python
import tensorflow as tf
ds = (tf.data.Dataset.from_tensor_slices((X_tr, y_tr))
        .shuffle(10_000).batch(256)
        .prefetch(tf.data.AUTOTUNE))
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(1, activation="sigmoid")])
model.compile(optimizer="adam", loss="binary_crossentropy",
    metrics=[tf.keras.metrics.AUC(name="auc")])
cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_auc", mode="max", patience=3,
    restore_best_weights=True)
model.fit(ds, validation_data=(X_val, y_val),
          epochs=30, callbacks=[cb])
model.export("gs://ml-models/churn/1")
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

L1 Import TensorFlow (Keras comes with it).

L2-4 **tf.data** pipeline: slice arrays, shuffle, batch, then **prefetch** so the accelerator never waits.

L5-8 Keras Sequential: dense layer, dropout against overfitting, sigmoid output for a probability.

L9-10 Compile: optimizer, loss and AUC metric (better than accuracy for imbalance).

L11-13 **EarlyStopping** watches val_auc; stops after 3 flat epochs and restores the best weights.

L14-15 Train with validation every epoch; the callback babysits the run.

L16 Export a SavedModel to GCS; Vertex AI or TF Serving can load it directly.

INTERVIEW CODE QUESTION

- GPU utilization is only 30%. The model is fine. What is wrong with this input pipeline, and how do you fix it?

THE INTERVIEW CODE

```python
# SLOW: no parallelism, no prefetch
ds = ds.map(parse).batch(256)

# FAST
ds = (ds.map(parse,
          num_parallel_calls=tf.data.AUTOTUNE)
        .cache()
        .shuffle(10_000)
        .batch(256)
        .prefetch(tf.data.AUTOTUNE))
```

INTERVIEW CODE | STEP BY STEP

1) An idle GPU means it is starved by the input pipeline, not a model problem.
2) **num_parallel_calls=AUTOTUNE** runs **parse** across many CPU cores.
3) **cache()** after the expensive deterministic map reuses results across epochs (if it fits).
4) Shuffle BEFORE batch; shuffling batches instead of rows is a classic mistake.
5) **prefetch** overlaps preprocessing with the training step.
6) Verify with the TensorBoard Profiler: look for the 'input bound' warning.

6 TensorFlow

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: Predictions from the live endpoint disagree with the offline notebook.

1) Training/serving skew: the transforms must travel with the model.
2) Put preprocessing inside the model (Keras preprocessing layers) or use TF Transform.
3) Export one SavedModel with a serving signature that accepts raw features.
4) Test: raw sample to endpoint output must match the offline prediction within tolerance.
5) Validate the input schema in the pipeline with TFDV.

SCENARIO Q2 | PRODUCTION

Situation: A Vertex AI endpoint is expensive because night traffic is nearly zero.

1) Autoscale with a sensible min replica count; Vertex endpoints keep at least one replica.
2) For bursty or tiny traffic, consider Cloud Run or batch prediction instead.
3) Right-size the machine type; quantize or shrink the model.
4) Move non-urgent scoring to batch prediction jobs.
5) Track cost per 1,000 predictions as a KPI and review it monthly.

SCENARIO Q3 | PRODUCTION

Situation: Training works, but loading the SavedModel in the serving container fails on a custom layer.

1) Register custom objects for serialization, or export traced serving functions.
2) Test the exported model in a clean container before deploying.
3) Pin TF / Keras versions across training and serving; add a CI smoke test (load, predict one sample, compare).

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Build a TFRecord + tf.data pipeline reading from GCS and reach over 80% accelerator utilization.
2) Train with `MirroredStrategy`, then multi-worker; compare throughput and cost.
3) Create a TFX / Vertex pipeline: validate, transform, train, evaluate, conditional push.
4) Quantize to TFLite INT8; report size, latency and accuracy drop.
5) Set up a canary: 10% traffic to the new SavedModel on Vertex AI with auto rollback on metric drop.

KEY POINTS TO REMEMBER IN INTERVIEWS

- **tf.data** order: map, cache, shuffle, batch, prefetch.
- **SavedModel** is the deployment format; know serving signatures.
- `@tf.function` traces once: Python side effects run only at trace time.
- Use mixed precision and a distribution strategy to scale.
- TF vs PyTorch: TF is traditionally strong on serving, TPU and edge; PyTorch on research flexibility. Both are production-ready.
- Ship preprocessing with the model to avoid skew.

COMMON MISTAKES THAT COST OFFERS

- Shuffling after batching.
- No prefetch in the input pipeline.
- Different preprocessing online than offline.
- Shipping notebooks without an exported serving signature.

7 PySpark

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture One chef must cook for 10,000 wedding guests. He will never finish.

Trouble One computer cannot hold or process data that huge.

Helper Hire 100 chefs. The head chef (driver) splits the guest list into tables (partitions). Each line chef (executor) cooks one table.

In ML PySpark splits a giant data job across many computers so it finishes fast.

WHY THIS LIBRARY

- Process data far bigger than one machine's RAM.
- Same DataFrame idea as pandas, but distributed and lazy.
- Native on AWS (EMR, Glue) and GCP (Dataproc, Serverless Spark).
- MLlib for distributed ML; pandas UDFs for custom Python.

WHEN TO USE / WHEN TO SKIP

- **Use:** hundreds of GB to TB+, huge joins, feature pipelines.
- **Use:** batch scoring millions of rows with a trained model.
- **Skip:** small data (overhead beats benefit) and low-latency serving.

HOW TO USE IT

1) Get a **SparkSession** (EMR, Glue, Dataproc provide it).
2) Read Parquet from S3 / GCS with partition pruning.
3) Transform with the DataFrame API; avoid Python UDFs when you can.
4) Write partitioned Parquet; tune shuffle partitions.

EXPERT VIEW

- The cheapest shuffle is the one you avoided. Filter early, broadcast small tables, watch for skew.

ADVANTAGES

- Scales horizontally: add nodes, not rewrites.
- Lazy evaluation plus the Catalyst optimizer plans efficiently.
- Fault tolerant: failed tasks are retried automatically.
- Same code runs on EMR, Glue, Dataproc and Databricks.

REAL-WORLD EXAMPLES (illustrative)

- **Amazon-style:** EMR / Glue Spark jobs build features from huge order and clickstream tables.
- **Google:** Dataproc Serverless Spark with the BigQuery connector for large feature jobs.
- **Meta-scale:** billions of event rows rolled up into daily user features.
- **Tesla-style:** fleet telemetry aggregated at scale before training.
- **Bank:** end-of-day transaction aggregation for fraud and AML features.

PRODUCTION TASKS | AWS + GCP

- Nightly feature generation writing partitioned Parquet to S3 / GCS (offline feature store).
- Distributed batch inference with `pandas_udf` / `mapInPandas`.
- Data-quality checks (counts, null rates) before handing data to training.
- Scheduled by Airflow: cluster created, used, destroyed per run.

COST + TIME SAVED

- Transient EMR / Dataproc clusters with spot nodes: pay only while the job runs.
- Partitioning and column pruning mean scanning a fraction of the data.
- Right-sized shuffle partitions often cut runtime and cluster hours dramatically.

7 PySpark

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```
1  from pyspark.sql import SparkSession, functions as F, Window
2  spark = (SparkSession.builder.appName("txn_features")
3           .getOrCreate())
4  tx = (spark.read.parquet("s3://ml-bank/txns/")
5          .filter(F.col("dt") == "2026-10-08")
6          .select("cust_id", "ts", "amount"))
7  w = Window.partitionBy("cust_id").orderBy("ts")
8  tx = tx.withColumn("gap_s", F.col("ts").cast("long")
9          - F.lag("ts").over(w).cast("long"))
10 feat = (tx.groupBy("cust_id")
11         .agg(F.count("*").alias("n_txn"),
12              F.avg("amount").alias("avg_amt"),
13              F.min("gap_s").alias("min_gap_s")))
14 feat.write.mode("overwrite").parquet("s3://ml-bank/feat/")
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

L1 **F** is the usual alias for **pyspark.sql.functions**; **Window** enables per-customer ordering.

L2-3 Entry point. On EMR / Glue / Dataproc it already exists; **getOrCreate** reuses it.

L4-6 Read Parquet, filter on **dt** (a partition column) so Spark reads one day, not the table, and select only needed columns.

L7 Window: per customer, rows ordered by time. It causes a shuffle: costly.

L8-9 **gap_s** = seconds since the previous transaction: a classic fraud 'velocity' feature.

L10-13 Aggregate per customer: count, average, shortest gap.

L14 Nothing ran until this line! Spark is lazy; the write triggers the whole plan.

INTERVIEW CODE QUESTION

- A Spark job has 200 tasks. 199 finish in 20 seconds but one runs for 40 minutes. What is happening and how do you fix it?

THE INTERVIEW CODE

```
# Skewed key: a few cust_id hold most rows
salted = (big.withColumn("salt",
              (F.rand() * 10).cast("int"))
    .groupBy("cust_id", "salt")
    .agg(F.sum("amount").alias("part")))
final = (salted.groupBy("cust_id")
    .agg(F.sum("part").alias("amount")))
```

INTERVIEW CODE | STEP BY STEP

1) Symptom of **data skew**: one partition holds a hot key (say, a corporate account).
2) Confirm in the Spark UI: Stages, then max vs median task duration.
3) **Salting** splits the hot key into 10 sub-keys so 10 tasks share the work.
4) Stage 1 builds partial sums in parallel; stage 2 merges just 10 rows per key.
5) For skewed joins: enable AQE skew join or broadcast the small side.
6) Adding executors will not help: the one hot task still runs alone.

7 PySpark

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: A daily EMR feature job grew from 1 hour to 6 hours as data doubled.

1) Open the Spark UI: find the slow stage; is it skew, spill or too many small files?
2) Prune early: filter partitions and select columns before joins.
3) Broadcast small dimension tables (`F.broadcast`) to avoid shuffles.
4) Tune `spark.sql.shuffle.partitions` or enable AQE to right-size shuffles.
5) Compact small files; use spot task nodes to cut cost.

SCENARIO Q2 | PRODUCTION

Situation: Score 300M customers nightly with a scikit-learn model. How?

1) Broadcast the model (joblib bytes) to the executors once.
2) Use `pandas_udf` or `mapInPandas` so scoring runs on batches, not row by row.
3) Repartition so each executor gets balanced chunks; tune batch size.
4) Write scores partitioned by date to S3 / GCS for downstream jobs.
5) Compare with SageMaker Batch Transform or Vertex batch prediction on cost.

SCENARIO Q3 | PRODUCTION

Situation: A job writes 200,000 tiny Parquet files and downstream reads crawl.

1) Too many small files: control output partitions with `coalesce` / `repartition` before the write.
2) Aim for files of roughly 128 MB to 1 GB; `partitionBy` on low-cardinality columns (date), never user_id.
3) Schedule a compaction job, or use Delta / Iceberg tables with auto-compaction.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Build an incremental feature pipeline (daily partitions, idempotent overwrite by dt) on EMR Serverless or Dataproc Serverless.
2) Diagnose and fix a skewed join with AQE, broadcast and salting; show before/after stage times.
3) Implement `pandas_udf` batch scoring for a joblib model; measure rows per second and cost.
4) Write a small-files compactor job and schedule it in Airflow.
5) Add data-quality checks (row counts, null %, duplicates) that fail the DAG on breach.

KEY POINTS TO REMEMBER IN INTERVIEWS

- Transformations are lazy; **actions** (write, count, collect) trigger the work.
- Avoid `collect()` / `toPandas()` on big data: the driver runs out of memory.
- Wide transformations cause shuffles: the expensive part.
- Prefer built-in functions over Python UDFs (Catalyst can optimize them).
- **repartition** vs **coalesce**; cache only what you reuse.
- Broadcast small tables; watch for skew and small files.

COMMON MISTAKES THAT COST OFFERS

- `collect()` on large data.
- A Python UDF where a built-in function exists.
- The default 200 shuffle partitions for every job.
- Caching everything 'just in case'.

8 Airflow

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture A train station timetable. Each train leaves only after the one before it has arrived.

Trouble When a train breaks down, someone must notice and fix it, maybe at 3 AM.

Helper The station master (Airflow) runs trains on time, retries a broken one, and shows every train as green or red. The timetable is the DAG.

In ML Airflow runs get data, train, check, deploy in order, on schedule, and retries failures.

WHY THIS LIBRARY

- Orchestrates multi-step ML pipelines with schedules, retries and dependencies.
- Everything is Python code: reviewable, versioned, testable.
- Providers for AWS (S3, EMR, SageMaker) and GCP (BigQuery, Dataproc, Vertex AI).
- A UI for monitoring, re-running failed tasks and backfills.

WHEN TO USE / WHEN TO SKIP

- **Use:** daily or hourly retraining, feature pipelines, batch scoring, quality workflows.
- **Use:** work that spans services (Spark, SageMaker, Slack).
- **Skip:** streaming or real-time inference, and heavy compute inside the Airflow worker.

HOW TO USE IT

1) Define the DAG: schedule, start_date, catchup, retries.
2) Create tasks with operators or **@task** functions.
3) Set dependencies with **>>**; pass only small metadata via XCom.
4) Deploy on MWAA (AWS) or Cloud Composer (GCP); alert on failure.

ADVANTAGES

- Retries, SLAs, backfills and alerts out of the box.
- Idempotent runs keyed by logical date: safe to re-run.
- Managed options (MWAA, Cloud Composer) remove infrastructure pain.
- Huge operator ecosystem, plus custom operators when needed.

REAL-WORLD EXAMPLES (illustrative)

- **Airbnb:** created Airflow; it is now an Apache project used widely.
- **Amazon:** MWAA orchestrates Glue, then SageMaker training, then endpoint update.
- **Google:** Cloud Composer triggers BigQuery and Vertex AI jobs.
- **Tesla-style:** fleet data DAGs: ingest, clean, train, evaluate.
- **Bank:** end-of-day risk features, model scoring and audit reports with logs.

PRODUCTION TASKS | AWS + GCP

- Retraining DAG: extract, validate, features (Spark), train, evaluate, approve, deploy.
- Data-quality gates with branching: skip training when checks fail.
- Slack / PagerDuty alerts on failure and on SLA misses.
- Backfills to reprocess history after a bug fix.

COST + TIME SAVED

- Replaces cron scripts and manual reruns: engineers stop babysitting nightly jobs.
- Transient clusters created and deleted by the DAG: no idle clusters burning money.
- Retries plus alerts turn silent failures into minute-level fixes.

EXPERT VIEW

- Airflow schedules; other systems compute. Keep DAG files light and tasks idempotent.

8 Airflow

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```python
from datetime import datetime, timedelta
from airflow.decorators import dag, task
args = {"owner": "ml-platform", "retries": 2,
        "retry_delay": timedelta(minutes=5)}
@dag(schedule="0 2 * * *", start_date=datetime(2026,10,1),
     catchup=False, default_args=args, tags=["ml"])
def churn_retrain():
    @task
    def features(ds=None):
        return f"s3://ml/features/dt={ds}/"
    @task
    def train(path):
        return run_training_job(path)  # model URI
    @task
    def gate(uri):
        assert evaluate(uri) > 0.85, "AUC below gate"
    gate(train(features()))
churn_retrain()
```

INTERVIEW CODE QUESTION

- Why is heavy code at the top level of a DAG file dangerous, and what do catchup and idempotency mean?

THE INTERVIEW CODE

```python
# BAD: runs on EVERY scheduler parse
df = pd.read_csv("s3://big/file.csv")

# GOOD: heavy work inside tasks
@task
def load():
    return pd.read_csv("s3://big/file.csv").shape

# backfill a past window (CLI)
# airflow dags backfill churn_retrain
#   -s 2026-09-01 -e 2026-09-07
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

L1-2 Imports: time helpers and the TaskFlow decorators.

L3-4 Every task retries twice, 5 minutes apart: handles flaky APIs and spot interruptions.

L5-6 Runs daily at 02:00. **catchup=False** means no automatic runs for every missed day at deploy.

L7 The function body describes the DAG structure, not the work itself.

L8-10 Task 1: **ds** is the logical date, so each run reads its own partition (idempotent).

L11-13 Task 2: trains and returns only a small URI through XCom (never a DataFrame). **run_training_job** is your helper that wraps SageMaker / Vertex.

L14-16 Task 3: quality gate. A failing assert stops the pipeline, so no bad model ships.

L17 Wiring: features, then train, then gate. Airflow infers the order from the data passing.

L18 Calling the function registers the DAG with the scheduler.

INTERVIEW CODE | STEP BY STEP

1) The scheduler re-parses DAG files every few seconds; top-level code runs each time and slows or crashes it.
2) Put heavy imports and work inside tasks.
3) **Idempotent** task: same logical date gives the same output, so a re-run is safe (overwrite the partition).
4) **catchup=True** would create a run for every interval since start_date.
5) Backfill re-runs history after a bug fix, using logical dates.
6) XCom is for small metadata (paths, IDs), not data: it lives in the metadata DB.

8 Airflow

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: The nightly training DAG fails at task 5 of 8 at 3 a.m.

1) Design for it: retries with backoff, failure callbacks to Slack / PagerDuty, a named owner.
2) Re-run only the failed task and downstream (Clear), not the whole DAG.
3) Idempotent tasks make re-runs safe: no duplicate outputs.
4) Root-cause from task logs, then add a check so it fails earlier next time.
5) After the incident: add an SLA and a runbook link to the alert.

SCENARIO Q2 | PRODUCTION

Situation: 100+ DAGs, the scheduler is slow and tasks queue for minutes.

1) Check DAG parse time: remove heavy imports and API calls at the top level.
2) Use pools and `max_active_tasks` to protect hot resources (DB, Spark cluster).
3) Pick a scalable executor (Celery / Kubernetes) and right-size workers.
4) Offload heavy compute to EMR / Dataproc / SageMaker; Airflow only orchestrates.
5) Use sensors in reschedule or deferrable mode to free worker slots.

SCENARIO Q3 | PRODUCTION

Situation: Someone used `datetime.now()` inside a task, and backfills produced wrong data.

1) Use the logical date (`ds`, data interval) so reruns reproduce the same window.
2) Never depend on wall-clock time inside tasks.
3) Test with `airflow dags test` for a past date; document the contract: one logical date, one output partition.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Design a retrain-evaluate-deploy DAG with branching: promote only if the challenger beats the champion.
2) Add Slack / PagerDuty failure callbacks and SLA alerts; write the runbook.
3) Create a transient EMR / Dataproc cluster per run (create, submit, delete with `trigger_rule=all_done`).
4) Write DAG integrity tests (import errors, cycles, tags, owners) and run them in CI.
5) Use dynamic task mapping to train one model per region with failure isolation.

KEY POINTS TO REMEMBER IN INTERVIEWS

- A DAG has no cycles; tasks should be **idempotent** and atomic.
- No heavy work at the top level of DAG files.
- XCom: small metadata only.
- Use the logical date (`ds`, data interval), not `datetime.now()`.
- Know `catchup`, `retries`, `depends_on_past`, `trigger_rule`.
- Airflow orchestrates; Spark, SageMaker and Vertex do the heavy compute.

COMMON MISTAKES THAT COST OFFERS

- Heavy code at the top level of DAG files.
- Passing DataFrames through XCom.
- Non-idempotent tasks (append without dedup).
- No alerts on failure.

9 MLflow

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture A science lab where every experiment goes into a labeled folder: the recipe, the ingredients and a photo of the result.

Trouble After 30 experiments, nobody remembers which one gave the best result.

Helper MLflow fills the folder automatically, so months later you can recreate the exact same cake.

In ML It records settings (parameters), scores (metrics) and models, and keeps the approved one.

WHY THIS LIBRARY

- Tracks params, metrics, artifacts and code version for every run.
- **Model Registry**: versions and aliases (champion, challenger).
- Framework-agnostic: scikit-learn, XGBoost, PyTorch, TensorFlow via flavors.
- Reproducibility and an audit trail: vital for banks and regulated teams.

WHEN TO USE / WHEN TO SKIP

- **Use:** any team running more than a handful of experiments or retrains.
- **Use:** promotion workflows, lineage, champion vs challenger.
- **Skip:** using it as a feature store or data-versioning tool. It is not one.

HOW TO USE IT

1) `set_tracking_uri` to a shared server (backed by RDS / Cloud SQL + S3 / GCS).
2) `log_params`, `log_metric`, `log_model` inside a run.
3) Register the best model; point the alias **champion** at it.
4) Serving or batch code loads **models:/name@champion**.

ADVANTAGES

- One source of truth for experiments and models.
- Easy integration: `autolog` covers popular libraries.
- Self-host on AWS / GCP or use managed options (SageMaker MLflow, Databricks).
- Open source: no vendor lock-in.

REAL-WORLD EXAMPLES (illustrative)

- **Databricks:** created MLflow; adopted widely across tech and banking.
- **Amazon:** SageMaker offers managed MLflow tracking servers.
- **Google:** GCP teams often run MLflow on GKE / Cloud Run with Cloud SQL + GCS.
- **Tesla-style:** hundreds of model variants compared with clean lineage.
- **Bank:** model risk management: who trained what, on which data, with which params.

PRODUCTION TASKS | AWS + GCP

- Training scripts log runs and artifacts automatically.
- CI/CD reads the registry alias and deploys only approved versions.
- Automated challenger vs champion metric comparison.
- Lineage tags: git SHA, data version, pipeline run ID.

COST + TIME SAVED

- Stops the 'nobody remembers which run was good' re-runs, saving GPU-hours.
- Audits shrink from days of detective work to one registry page.
- Fewer bad deployments means fewer incidents and rollbacks.

EXPERT VIEW

- If a result is not logged with its code version and data version, as far as the audit is concerned it never happened.

9 MLflow

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```python
import mlflow, mlflow.sklearn
from mlflow import MlflowClient
mlflow.set_tracking_uri("https://mlflow.bank.internal")
mlflow.set_experiment("credit_risk")
with mlflow.start_run(run_name="rf_v7") as run:
    mlflow.log_params({"n_estimators": 200, "max_depth": 8})
    model.fit(X_tr, y_tr)
    mlflow.log_metric("val_auc", auc)
    mlflow.set_tag("git_sha", GIT_SHA)
    mlflow.sklearn.log_model(model, "model")
mv = mlflow.register_model(
    f"runs:/{run.info.run_id}/model", "credit_risk_rf")
if auc > champion_auc:
    MlflowClient().set_registered_model_alias(
        "credit_risk_rf", "champion", mv.version)
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

L1-2 Core tracking API plus the client used for registry operations.

L3-4 Point at the shared server (never localhost) and group runs in an experiment.

L5 Everything in the **with** block is one run; it closes cleanly even on errors.

L6-9 Log what you tried (params), what you got (metric) and which code (git SHA tag).

L10 Save the model in a standard flavor so any service can load it.

L11-12 Register that exact run's model as a new version of **credit_risk_rf**.

L13-15 The gate: only if it beats the current champion, move the **champion** alias to the new version.

INTERVIEW CODE QUESTION

- How does a batch job always load the production model, so deploys and rollbacks never need a code change?

THE INTERVIEW CODE

```python
import mlflow.pyfunc
model = mlflow.pyfunc.load_model(
    "models:/credit_risk_rf@champion")
preds = model.predict(batch_df)

# rollback = move the alias, no redeploy
client.set_registered_model_alias(
    "credit_risk_rf", "champion", 6)
```

INTERVIEW CODE | STEP BY STEP

1) **models:/name@alias** resolves to a concrete version at load time.
2) **pyfunc** gives one **predict()** interface for sklearn, XGBoost, PyTorch and more.
3) Promotion = move the alias. Application code never changes.
4) Rollback = move the alias back to version 6: seconds, not a redeploy.
5) Log which model version served each batch: that is your audit trail.
6) Pin the model's environment (requirements / conda) so dependencies cannot surprise you.

9 MLflow

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: A regulator asks which data, code and parameters produced the model that scored customers last March.

1) Registry version leads to its source run: params, metrics, git SHA, data version tag.
2) That is why every training run must tag the git SHA and a dataset snapshot ID.
3) Artifacts (model, plots, SHAP) sit immutably in S3 / GCS.
4) You answer in minutes, with a lineage page you can screenshot.
5) Add approval tags or comments in the registry for the sign-off trail.

SCENARIO Q2 | PRODUCTION

Situation: The MLflow UI is slow, and runs vanished after a single EC2 server restarted.

1) Never use the local file store or SQLite in production.
2) Backend store: managed Postgres (RDS / Cloud SQL). Artifacts: S3 / GCS.
3) Run the tracking server behind a load balancer with auth (SSO / IAM).
4) Or use managed MLflow (SageMaker) and stop operating it yourself.
5) Add backups and a retention policy; archive old runs.

SCENARIO Q3 | PRODUCTION

Situation: Two teammates both promoted a model on the same day. Which one is live?

1) Aliases make the live version explicit: exactly one version holds **champion**.
2) Restrict who can move aliases (permissions, or CI-only promotion).
3) Log promotion events (who, when, metric evidence) and deploy by alias so all environments agree.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Stand up MLflow on AWS (ECS/EKS + RDS + S3) or GCP (Cloud Run + Cloud SQL + GCS) with authentication.
2) Create an automated champion/challenger promotion job using aliases and metric thresholds.
3) Write a decorator that logs git SHA, data version and environment hash for every training run.
4) Build a nightly report comparing live metrics with the registered model's offline metrics.
5) Add a CI check: deployment fails if the model version lacks signature, tags or approval.

KEY POINTS TO REMEMBER IN INTERVIEWS

- Track params, metrics, artifacts and **tags** (git SHA, data version).
- Registry **aliases** (champion) replace the older stages in newer versions.
- Log the model **signature** and an input example to catch schema errors early.
- Remote server: database backend + object-store artifacts.
- `autolog` is convenient; log critical metrics explicitly.
- MLflow tracks experiments; it does not orchestrate. That is Airflow's job.

COMMON MISTAKES THAT COST OFFERS

- Logging only the final metric and no params.
- No git SHA or data version on the run.
- A local file store in production.
- Treating the registry as a data store.

10 boto3

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture A TV remote for a giant hotel. One button opens doors, another turns on lights, another orders room service.

Trouble Clicking through the AWS website is slow, and nobody can repeat it exactly.

Helper AWS is the hotel. S3 is the storage room. SageMaker is the training gym. boto3 is the remote, held by Python.

In ML Write the button presses once and the script never forgets a step.

WHY THIS LIBRARY

- The official AWS SDK for Python: nearly every AWS service in one library.
- Automates S3 data movement, SageMaker jobs, endpoints and Lambdas.
- Infrastructure steps become versioned code.
- IAM roles give secure, keyless access in production.

WHEN TO USE / WHEN TO SKIP

- **Use:** SageMaker jobs and endpoints, S3 I/O, Secrets Manager, Lambda, Athena, Glue.
- **Use:** glue code between pipeline stages.
- **Skip:** large-scale infra provisioning (Terraform / CDK) and heavy data science.

HOW TO USE IT

1) Rely on an IAM role (no hard-coded keys); set the region.
2) Create a client: `boto3.client("s3")`.
3) Call the API; handle pagination and `ClientError`.
4) Log request IDs and keep scripts idempotent.

EXPERT VIEW

- Automate anything you do twice in the console, and make the script safe to run twice.

ADVANTAGES

- Built-in retries, paginators and waiters.
- New AWS features reach the SDK quickly.
- Works the same on laptop, Lambda, EC2, ECS and SageMaker.
- The credential chain picks up the IAM role automatically.

REAL-WORLD EXAMPLES (illustrative)

- **Amazon:** ML platform teams script SageMaker training and endpoints with it.
- **Google-style multi-cloud:** boto3 for the AWS side next to google-cloud libraries.
- **Meta-scale:** parallel multipart S3 transfers for very large files.
- **Tesla-style:** S3 landing zones for telemetry, Lambda triggers start processing.
- **Bank:** KMS-encrypted S3, Secrets Manager and CloudTrail audit logs.

PRODUCTION TASKS | AWS + GCP

- Start SageMaker training, poll with waiters, create or update endpoints.
- Blue/green endpoint updates with automatic rollback scripts.
- Housekeeping: delete old endpoints and models, tag resources, report costs.
- S3 multipart uploads, presigned URLs and lifecycle rules.

COST + TIME SAVED

- Scripts that delete idle endpoints and stop dev notebooks nightly commonly save thousands per month.
- S3 lifecycle and Intelligent-Tiering rules (set by script) trim storage bills.
- Replaces repetitive console clicking: hours per week back to the team.

10 boto3

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```python
import boto3, json
from botocore.exceptions import ClientError
s3 = boto3.client("s3")
rt = boto3.client("sagemaker-runtime")
def latest_key(bucket, prefix):
    pages = s3.get_paginator("list_objects_v2").paginate(
        Bucket=bucket, Prefix=prefix)
    objs = [o for p in pages for o in p.get("Contents", [])]
    return max(objs, key=lambda o: o["LastModified"])["Key"]
try:
    r = rt.invoke_endpoint(EndpointName="fraud-prod",
        ContentType="application/json",
        Body=json.dumps({"instances": rows}))
    scores = json.loads(r["Body"].read())
except ClientError as e:
    print(e.response["Error"]["Code"]); raise
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

L1-2 **boto3** plus **ClientError**, the exception you actually catch for AWS API failures.

L3-4 Clients for S3 and SageMaker Runtime. No keys in code: the IAM role is picked up automatically.

L5-9 Find the newest file under a prefix. A **paginator** is a must: list_objects_v2 returns at most 1,000 keys per call.

L10-13 Call a live endpoint with JSON rows. **ContentType** must match what the container expects.

L14 The body is a stream: **.read()**, then parse the JSON.

L15-16 Catch AWS errors (throttling, validation, model errors), log the code and **re-raise** so the pipeline sees the failure.

INTERVIEW CODE QUESTION

- Upload a 5 GB file to S3 reliably, then give a customer temporary download access. Show the code and explain it.

THE INTERVIEW CODE

```python
from boto3.s3.transfer import TransferConfig
cfg = TransferConfig(multipart_threshold=64*2**20,
                     max_concurrency=10)
s3.upload_file("big.parquet", "ml-data",
               "in/big.parquet", Config=cfg)
url = s3.generate_presigned_url("get_object",
    Params={"Bucket": "ml-data",
            "Key": "in/big.parquet"},
    ExpiresIn=900)
```

INTERVIEW CODE | STEP BY STEP

1) A single PUT is capped at 5 GB. **Multipart** splits the file into parts, uploaded in parallel and retried individually.
2) **TransferConfig**: threshold decides when to go multipart; concurrency sets parallel threads.
3) **upload_file** handles retries and part assembly; **upload_fileobj** is for streams.
4) A **presigned URL** is a temporary link signed with your role's credentials.
5) **ExpiresIn=900** is 15 minutes. Keep it short.
6) Security: private bucket, SSE-KMS and least-privilege IAM. Never make buckets public.

10 boto3

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: The cloud bill spiked: 40 forgotten SageMaker endpoints are running.

1) boto3 script: `list_endpoints`, check tags/owner and last invocation (CloudWatch).
2) Delete, or notify owners of, endpoints with zero invocations for N days.
3) Enforce tags (owner, project, ttl) at creation; run cleanup on a schedule (EventBridge + Lambda).
4) Add an AWS Budgets alert and a cost report by tag.
5) Report the savings monthly so the automation stays funded.

SCENARIO Q2 | PRODUCTION

Situation: A pipeline step fails randomly with **ThrottlingException**.

1) Throttling means too many API calls; it is normal at scale.
2) Use `Config(retries={"max_attempts": 10, "mode": "adaptive"})`.
3) Add exponential backoff with jitter in your own loops.
4) Make fewer calls: paginators, batch APIs, cached describe results.
5) Ask for quota increases only after you have removed waste.

SCENARIO Q3 | PRODUCTION

Situation: A Lambda calling SageMaker works locally but fails in AWS with AccessDenied.

1) Locally you used your own profile; Lambda uses its execution role.
2) Read the error: it names the denied action and resource. Grant least privilege for exactly those.
3) Verify with the IAM policy simulator and keep policies in infrastructure-as-code.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Write an endpoint 'janitor' (Lambda + EventBridge) that tags, flags and deletes idle SageMaker resources.
2) Automate a blue/green endpoint update with health checks and automatic rollback on error rate.
3) Build an S3 data-lake mover: parallel multipart, checksums, resumable, lifecycle rules.
4) Create a secrets/config loader (Secrets Manager + Parameter Store) with caching and rotation handling.
5) Write a Cost Explorer script grouped by tag and post it weekly to Slack.

KEY POINTS TO REMEMBER IN INTERVIEWS

- Never hard-code keys: use IAM roles or profiles.
- **client** (low-level) vs **resource** (higher-level): know the difference.
- Always paginate list calls; use waiters for async resources.
- Catch **ClientError** and inspect **Error.Code**.
- Set timeouts and retries through botocore `Config`.
- Least-privilege IAM, KMS encryption, CloudTrail logging.

COMMON MISTAKES THAT COST OFFERS

- Hard-coded access keys.
- Skipping pagination on list calls.
- Catching `Exception` and swallowing it.
- Public S3 buckets 'for convenience'.

11 Vertex AI SDK + KFP

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture Lego instructions. Each small brick is one step and the booklet gives the order.

Trouble Steps done by hand in a notebook change a little every time.

Helper A component is one brick: a small Python function in its own box (container). The pipeline is the booklet. Vertex AI builds it the same way every time.

In ML KFP defines the steps, Vertex AI runs them, and unchanged steps are skipped.

WHY THIS LIBRARY

- Managed training, tuning, endpoints and pipelines on GCP: no cluster babysitting.
- KFP pipelines are reproducible, cached and tracked with lineage.
- Integrates with BigQuery, GCS, Artifact Registry and IAM.
- One SDK for notebooks, CI/CD and schedulers.

WHEN TO USE / WHEN TO SKIP

- **Use:** GCP ML platforms, scheduled retraining, managed endpoints, batch prediction.
- **Use:** multi-step workflows that need caching and artifact lineage.
- **Skip:** tiny one-off scripts, or AWS-only shops (use SageMaker Pipelines).

HOW TO USE IT

1) `aiplatform.init(project, location, staging_bucket)`.
2) Write components with **@dsl.component**; chain them in **@dsl.pipeline**.
3) Compile to a YAML or JSON spec; submit with **PipelineJob**; schedule it.
4) Deploy the model to an **Endpoint** or run batch prediction.

ADVANTAGES

- Serverless: pay per job, nothing to patch.
- Step caching skips unchanged steps: faster reruns.
- Experiments, Metadata and Model Registry built in.
- Custom containers: bring scikit-learn, XGBoost, PyTorch or TF.

REAL-WORLD EXAMPLES (illustrative)

- **Google:** Kubeflow started at Google; Vertex Pipelines runs KFP and TFX.
- **Amazon:** the AWS equivalent is SageMaker Pipelines: same concepts, different SDK.
- **Meta-scale:** reusable pipeline components let many teams share steps.
- **Tesla-style:** GPU / TPU custom jobs launched on demand for training.
- **Bank:** private endpoints in a VPC, CMEK encryption, audit logs on every deployment.

PRODUCTION TASKS | AWS + GCP

- Pipeline: BigQuery extract, validate, train, evaluate, conditional deploy.
- Scheduled runs under a service account, with failure alerts.
- Model Registry versions and endpoint traffic split for canaries.
- Batch prediction to BigQuery / GCS for nightly scoring.

COST + TIME SAVED

- You pay only while steps run; caching skips repeated steps.
- Spot VMs for training steps and right-sized machines per component.
- Autoscaled endpoints; auto-undeploy of dev endpoints.

EXPERT VIEW

- A pipeline without a quality gate is just a faster way to ship bad models.

11 Vertex AI SDK + KFP

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```python
from kfp import dsl, compiler
from google.cloud import aiplatform as aip
@dsl.component(packages_to_install=["scikit-learn"])
def train(data: str, model: dsl.Output[dsl.Model]) -> float:
    ...  # fit, save to model.path, return val_auc
@dsl.pipeline(name="churn-retrain")
def pipe(data: str):
    t = train(data=data)
    with dsl.If(t.output > 0.85):
        deploy(model=t.outputs["model"])
compiler.Compiler().compile(pipe, "pipe.json")
aip.init(project="ml-prod", location="us-central1",
         staging_bucket="gs://ml-staging")
job = aip.PipelineJob(display_name="churn-retrain",
    template_path="pipe.json", enable_caching=True,
    parameter_values={"data": "gs://ml/churn.parquet"})
job.submit(service_account=SA)
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

L1-2 **kfp** authors the pipeline; the **aiplatform** SDK runs it on Vertex.

L3-5 A **component** is a function that runs in its own container; pip packages install per component. It takes typed inputs and returns the validation AUC.

L6-8 Pipeline wiring: calling **train()** adds a step to the graph. It does not run now.

L9-10 Conditional deploy: only if AUC is above 0.85. The quality gate lives inside the pipeline. **deploy** is your own component.

L11 Compile to a portable pipeline spec (YAML or JSON): the file you version in Git.

L12-13 Initialize project, region and staging bucket (artifacts land here).

L14-16 Create the job; **enable_caching** skips unchanged steps; pass parameters.

L17 Submit under a least-privilege service account, never a person's credentials.

INTERVIEW CODE QUESTION

- Deploy a new model to an existing Vertex AI endpoint with a 10% canary and explain how rollback works.

THE INTERVIEW CODE

```python
ep = aip.Endpoint(ENDPOINT_ID)
model = aip.Model(MODEL_RESOURCE_NAME)
model.deploy(endpoint=ep,
    machine_type="n1-standard-4",
    min_replica_count=1,
    max_replica_count=5,
    traffic_percentage=10)
```

INTERVIEW CODE | STEP BY STEP

1) Reference the existing endpoint and the newly registered model.
2) **traffic_percentage=10** sends 10% to the new model; 90% stays on the old one.
3) **min / max replicas** control autoscaling: cost versus latency.
4) Watch error rate, latency and the business metric in Cloud Monitoring for a window.
5) Healthy: shift to 100%. Unhealthy: set the new model's traffic to 0 (rollback).
6) Automate the rollback script and alert. Do not rely on a Slack message at midnight.

11 Vertex AI SDK + KFP

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: A retraining pipeline takes 3 hours; 2.5 of them are unchanged data prep.

1) Turn on pipeline caching so unchanged components with the same inputs are reused.
2) Split prep from training: prep runs only when data changes.
3) Push heavy prep into BigQuery SQL instead of pandas inside a component.
4) Run independent steps in parallel (for example, per region).
5) Use spot VMs and right-size machines per component.

SCENARIO Q2 | PRODUCTION

Situation: A bank requires private networking and customer-managed keys for ML on GCP.

1) Run jobs and endpoints in a VPC (Private Service Connect / peering): no public IPs.
2) CMEK encryption for datasets, models and pipeline artifacts.
3) One service account per pipeline with least-privilege IAM; no user keys.
4) A VPC Service Controls perimeter to prevent data exfiltration.
5) Audit logs to the central SIEM; deployment needs an approval step in the pipeline.

SCENARIO Q3 | PRODUCTION

Situation: A scheduled pipeline silently trained on stale data because caching reused a component.

1) Caching is keyed on inputs: a constant path with changing data still hits the cache.
2) Pass the date or partition as an input parameter so the cache key changes.
3) Disable caching for extraction steps and add a freshness check that fails on old data.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Build a BigQuery, train, evaluate, conditional-deploy KFP pipeline with caching and lineage.
2) Schedule it weekly with a service account; alert on failure through Cloud Monitoring.
3) Implement canary plus automatic rollback for endpoint deployment based on live metrics.
4) Create a reusable components library, versioned in Artifact Registry.
5) Compare Vertex batch prediction vs an online endpoint on cost for 50M daily scores.

KEY POINTS TO REMEMBER IN INTERVIEWS

- Components are containerized: dependencies, inputs and outputs must be explicit.
- The pipeline definition runs at **compile time**; component code runs at **run time**.
- Caching is keyed on component + inputs. Know when to disable it.
- Service accounts and least privilege; never user credentials in jobs.
- Traffic split gives canary and rollback.
- SageMaker Pipelines is the AWS equivalent: know the mapping.

COMMON MISTAKES THAT COST OFFERS

- User credentials inside pipeline jobs.
- Giant monolithic components.
- No quality gate before deploy.
- Dev endpoints left deployed over the weekend.

12 FastAPI

PART 1 of 3 | Story, why, when, how, production value

STORY TIME | EVERYDAY ANALOGY

Picture A restaurant counter with a strict order form.

Trouble A bad order that reaches the kitchen wastes the chef's time.

Helper The cashier (Pydantic) checks every order and sends back incomplete ones at once. Good orders reach the kitchen (your model) and the answer goes out.

In ML FastAPI puts a model behind a web address and rejects bad input before the model sees it.

WHY THIS LIBRARY

- Fast, async-capable, with auto-generated OpenAPI / Swagger docs.
- Pydantic validation catches bad input early.
- Simple to containerize: Docker, then ECS / EKS / Cloud Run.
- Great for custom inference logic that managed endpoints do not allow.

WHEN TO USE / WHEN TO SKIP

- **Use:** real-time inference APIs, feature lookup services, ML microservices.
- **Use:** custom pre- and post-processing around a model.
- **Skip:** big batch scoring (Spark, batch jobs) or when a managed endpoint already does the job.

HOW TO USE IT

1) Define Pydantic request and response models.
2) Load the model **once** at startup (lifespan).
3) Create **/predict** and **/health** endpoints.
4) Run with uvicorn / gunicorn in Docker; add autoscaling and monitoring.

EXPERT VIEW

- The model is the easy part. Latency budgets, timeouts and rollbacks are the real serving work.

ADVANTAGES

- High performance (Starlette + uvicorn).
- Automatic validation and docs mean fewer integration bugs.
- Type hints make code self-documenting.
- Easy to test with `TestClient`.

REAL-WORLD EXAMPLES (illustrative)

- **Amazon:** containers on ECS / Fargate / EKS behind an ALB serving custom models.
- **Google:** Cloud Run services (autoscale, scale to zero) hosting inference APIs.
- **Meta-scale thinking:** stateless pods, horizontal scaling, strict latency budgets.
- **Tesla-style:** internal APIs behind model-evaluation dashboards.
- **Bank:** a fraud-check API called at payment time: p99 latency and audit logs matter.

PRODUCTION TASKS | AWS + GCP

- Model-serving microservice with /predict, /health, /ready, /metrics.
- Input validation, auth, rate limiting, structured logs.
- Shadow and canary deployments with CloudWatch / Prometheus metrics.
- CI/CD: build image, scan, push to ECR / Artifact Registry, roll out.

COST + TIME SAVED

- Cloud Run scale-to-zero and per-request billing suit spiky traffic.
- Right-sized containers with autoscaling avoid always-on big instances.
- Validation at the edge avoids wasting compute on junk requests.

12 FastAPI

PART 2 of 3 | Code lab + interview code question

CODE LAB | PRODUCTION SNIPPET

```python
from contextlib import asynccontextmanager
from fastapi import FastAPI
from pydantic import BaseModel, Field
import joblib
models = {}
@asynccontextmanager
async def lifespan(app):
    models["m"] = joblib.load("model.joblib")
    yield
app = FastAPI(lifespan=lifespan)
class Txn(BaseModel):
    amount: float = Field(gt=0)
    hour: int = Field(ge=0, le=23)
@app.post("/predict")
def predict(t: Txn):
    x = [[t.amount, t.hour]]
    p = models["m"].predict_proba(x)[0][1]
    return {"fraud_score": round(float(p), 4)}
@app.get("/health")
def health():
    return {"ok": "m" in models}
```

LINE BY LINE | EXPLAINED LIKE AN EXPERT

L1-4 Imports: lifespan helper, the app, Pydantic for validation, joblib to load the model.

L5 A module-level dict holds the model: loaded once, shared by every request.

L6-9 **Lifespan** loads the model at startup (NOT per request) and can clean up on shutdown.

L10 Create the app and attach the lifespan.

L11-13 Request schema: amount must be positive, hour 0-23. Bad input gets an automatic 422 error.

L14-15 POST /predict: FastAPI parses the JSON into **Txn** and validates it before your code runs.

L16-18 Build the feature row, get the fraud probability (class 1) and return rounded JSON.

L19-21 Health check for load balancers (ALB / Cloud Run): is the model loaded?

INTERVIEW CODE QUESTION

- The endpoint is written as async def, yet latency spikes under load. Why, and what do you change?

THE INTERVIEW CODE

```python
# BAD: blocks the event loop
@app.post("/predict")
async def predict(t: Txn):
    return model.predict(...)  # CPU-bound

# GOOD: plain def runs in a threadpool
@app.post("/predict")
def predict(t: Txn):
    return model.predict(...)
```

INTERVIEW CODE | STEP BY STEP

1) **async def** runs on the event loop; a blocking CPU call freezes ALL requests.
2) Plain **def** endpoints run in FastAPI's threadpool, so other requests keep moving.
3) For CPU-heavy models add worker processes: `uvicorn app:app --workers N`, sized to the CPU cores.
4) Use async only with awaitable I/O (async DB and HTTP clients).
5) Each worker loads its own model copy: watch memory times workers.
6) Measure p50 / p95 / p99 under load (Locust, k6) before and after.

12 FastAPI

PART 3 of 3 | Scenarios, practice tasks, key points

SCENARIO Q1 | PRODUCTION

Situation: A payment fraud API must answer within 100 ms at 2,000 requests per second.

1) Budget the 100 ms across network, feature fetch, model and response; measure each.
2) Keep the model in memory; use a small fast model; avoid per-request I/O.
3) Fetch features from a low-latency store (Redis, DynamoDB, Bigtable), not a warehouse.
4) Autoscale horizontally on CPU or latency; several workers per pod; keep warm instances.
5) Add timeouts and a fallback (rules engine) so payments never hang if the model is down.

SCENARIO Q2 | PRODUCTION

Situation: How do you roll out a new model version without customers noticing?

1) Package model and code in a versioned Docker image.
2) Shadow mode first: the new model scores, but outputs are only logged.
3) Canary: 5%, then 25%, then 100% (ALB weighted targets or Cloud Run traffic split).
4) Auto-rollback on error rate or score-distribution shift.
5) Return the model version in every response for debugging and audit.

SCENARIO Q3 | PRODUCTION

Situation: Memory climbs steadily on the inference pods until they restart.

1) Look for per-request model loading and growing caches or global lists.
2) Remember each worker holds its own model copy: reduce workers or shrink the model.
3) Profile with `tracemalloc` in staging; set pod memory limits and probes, but fix the root cause.

5 PRACTICE TASKS | 10+ YEARS LEVEL

1) Build /predict and /batch_predict with Pydantic schemas, auth and structured JSON logs.
2) Containerize with a slim image and health/readiness probes; deploy to Cloud Run and ECS Fargate.
3) Load test with Locust; tune workers and find max RPS at p99 under 100 ms.
4) Add Prometheus / CloudWatch metrics: latency histograms, error rate, prediction distribution.
5) Implement shadow plus canary routing with automatic rollback on a drift or error threshold.

KEY POINTS TO REMEMBER IN INTERVIEWS

- Load the model once at startup, never per request.
- **def** vs **async def**: blocking CPU work belongs in def (threadpool) or worker processes.
- Pydantic validation is the first line of defense; return clear 4xx errors.
- **/health** (alive) vs **/ready** (model loaded).
- Version your API and your model; log both.
- Set timeouts, limits and fallbacks; fail gracefully.

COMMON MISTAKES THAT COST OFFERS

- Loading the model per request.
- `async def` wrapped around blocking code.
- No limits on payload size.
- Logging raw sensitive payloads.

★ Python Code Key Points for ML Interviews

habits interviewers watch

PYTHON HABITS INTERVIEWERS WATCH

- Mutable default argument (`def f(x, acc=[])`) is a classic trap: default to `None`, create inside.
- `==` compares values, **is** compares identity. Use `is` only for None, True, False.
- List comprehension holds everything in RAM; a **generator** streams one item at a time.
- Assignment shares the object; `copy()` is shallow, `deepcopy()` is deep.
- Context managers (**with**) close files, sessions and connections even on errors.
- Type hints plus dataclass or Pydantic make interfaces self-documenting.
- Use `logging` with levels and request IDs, not `print()`, in production.

GIL AND PARALLELISM | ASKED EVERY TIME

- **Threads:** great for I/O waits (S3 calls, APIs). The GIL stops two threads running Python bytecode at once.
- **Processes** (`multiprocessing`, `joblib`): real CPU parallelism, but you pay serialization and memory.
- NumPy, Pandas and XGBoost release the GIL in C code, so threads can still help there.
- Beyond one machine: Spark, Ray, Dask or cloud batch jobs.

ERROR HANDLING AND RELIABILITY

- Catch specific exceptions, never a bare `except`; log, then re-raise.
- Retries with exponential backoff plus jitter for network calls; keep operations idempotent.
- Fail fast on bad input (validation); fail loudly in pipelines (non-zero exit code).
- Timeouts everywhere: a missing timeout is a hung pipeline.
- Config from environment variables or Parameter Store / Secret Manager, never hard-coded secrets.

TESTING AND PACKAGING

- pytest for unit tests; test transforms with tiny fixtures.
- Pin dependencies and the Python version in Docker.
- One shared package for preprocessing, used by training AND serving.
- pre-commit with black, ruff and mypy; CI runs tests before any deploy.

PERFORMANCE QUICK WINS

- Profile first (`cProfile`, `line_profiler`); fix the hot 5%.
- Vectorize; never `.append` row by row onto a big DataFrame.
- Cache pure functions with `functools.lru_cache`.
- Dict and set lookups are O(1); list search is O(n).

★ Library Chooser + Answer Recipe

say it clearly, under pressure

WHICH LIBRARY WHEN | QUICK CHOOSER

- Fast math on arrays → **NumPy**
- Tables that fit in RAM → **Pandas**
- Tables in the terabytes → **PySpark** (EMR, Glue, Dataproc)
- Classic ML, leak-proof pipeline → **scikit-learn**
- Best accuracy on tabular data → **XGBoost**
- Deep learning, research flexibility → **PyTorch**
- Deep learning, TPU / serving / edge → **TensorFlow**
- Schedules, retries, dependencies → **Airflow**
- Experiments and model registry → **MLflow**
- Automating AWS (S3, SageMaker) → **boto3**
- Managed GCP pipelines and endpoints → **Vertex AI SDK + KFP**
- Real-time model API → **FastAPI**

ANSWER ANY SCENARIO IN 60 SECONDS

1) **Clarify:** data size, latency, SLA, budget, compliance.
2) **Name the metric** that matters: PR-AUC, p99 latency, cost per 1,000 predictions.
3) **Propose the simplest design** that works, and mention the trade-off you rejected.
4) **Walk the pipeline:** data, train, validate, deploy, monitor.
5) **Call out failure modes:** skew, drift, retries, rollback.
6) **Close with the business result:** time saved, cost cut, risk lowered.

LAST-MINUTE CHECKLIST

- Say the **why** of a library before the **how**.
- Write code in small steps and narrate every line aloud.
- State time and memory complexity, and edge cases: empty input, NaN, duplicates.
- Ask: batch or real-time? Data size? Latency budget?

RED FLAGS INTERVIEWERS LISTEN FOR

- Random split on time-series data, or fitting the scaler on test rows.
- Hard-coded keys, no retries, no monitoring.
- Always-on GPU endpoints with no traffic.
- Notebook code shipped as production code.
- 'It worked on my machine' instead of a reproducible pipeline.